

Description Generator Testing Protocol

Date Created: December 26, 2024 **Purpose:** Standalone evaluation of the description generator before DreamCoder integration

Executive Summary

This protocol enables testing the description generator independently by: 1. Generating 201 diverse synthetic card game rules 2. Evaluating description quality on held-out rules 3. Measuring accuracy, consistency, and discrimination metrics 4. Comparing against baselines

Key Finding from Initial Testing: The current evaluation shows 0% accuracy due to **vocabulary mismatch** between the synthetic task generator’s expected primitives and the description generator’s output primitives. This needs to be addressed before meaningful evaluation.

Files Created

File	Purpose
src/description_generator/description_generator.py	Generator (1482 lines)
src/description_generator/synthesis_primitive_mapping.py	Primitive mapping (600 lines)
src/description_generator/synthetic_tasks.py	201 synthetic rules
src/description_generator/evaluation_harness.py	Fairness pipeline
docs/description_generator_evaluation_protocol.md	Detailed protocol specification

Running the Evaluation

Quick Test (5 minutes)

```
cd /Users/cankonuk/Documents/self-explanations-project/card-games-modelling/src
python3 description_generator/evaluation_harness.py --quick
```

Full Evaluation (30-60 minutes)

```
cd /Users/cankonuk/Documents/self-explanations-project/card-games-modelling/src
python3 description_generator/evaluation_harness.py --output results.json
```

With Holdout Validation (final validation only)

```
python3 description_generator/evaluation_harness.py --include-holdout --output final_results.json
```

Synthetic Rules Generated

Total: 201 rules across 5 complexity levels:

Level	Type	Count	Example
1	Atomic	50	“All cards same suit”
2	Comparison	22	“First and last same color”
3	Counting	47	“Exactly 3 unique ranks”

Level	Type	Count	Example
4	Pattern	29	“Sorted by rank”
5	Compositional	53	“Uniform suit AND sorted”

Evaluation Metrics

Primary Metrics

1. **Feature Capture Rate (FCR@3)**: Does the correct feature appear in top-3 descriptions?
2. **Primitive Jaccard Similarity**: Overlap between generated and expected primitives
3. **Consistency Score**: Same rule with different examples → same descriptions?
4. **Discrimination Score**: Different rules → different descriptions?

Baselines

1. **Random Baseline**: Select random descriptions from template pool
2. **Frequent Baseline**: Always output most common descriptions

Known Issues to Address

Issue 1: Vocabulary Mismatch

Problem: The synthetic task generator uses different primitive names than the description generator:

Synthetic Tasks Says	Description Generator Says
uniform, get_suit	n_unique_suits, all_same_suit
eq, 1	unique_count, eq

Solution Options: 1. Align synthetic task primitive names to match description generator 2. Add fuzzy matching in evaluation (e.g., `uniform all_same_*`) 3. Evaluate based on semantic similarity, not string matching

Issue 2: Rare Rules Hard to Sample

Some rules (e.g., “all same suit”) are very rare in random hands: - $P(\text{all 6 cards same suit}) = 0.2\%$ - May need 5000+ attempts to get 20 examples

Current Mitigation: `max_attempts=5000` in sampling functions

Issue 3: Multiple Valid Descriptions

A rule like “has pair” could be described as: - “two cards share a rank” - “fewer unique ranks than cards” - “repeated rank exists”

Evaluation Approach: Use Jaccard on primitives, not exact string match

Test Plan for Tomorrow

Step 1: Verify Components Work (15 min)

```
# Test synthetic task generator
python3 -c "
from description_generator.synthetic_tasks import generate_synthetic_rules
rules = generate_synthetic_rules(seed=42)
print(f'Generated {len(rules)} rules')
for r in rules[:5]:
    print(f' [{r.level}] {r.name}')
"

# Test description generator
python3 -c "
from description_generator.description_generator import DescriptionGenerator
from rules.cards import sample_hand
gen = DescriptionGenerator(n_baseline_samples=2000)
hand = sample_hand(6)
descs = gen.describe_hand(hand, top_k=3)
for d in descs:
    print(f' {d.text} (score: {d.score:.2f})')
"
```

Step 2: Run Quick Evaluation (10 min)

```
python3 description_generator/evaluation_harness.py --quick
```

Expected Output: Should show ~0% accuracy (vocabulary mismatch)

Step 3: Fix Vocabulary Mismatch (30 min)

Edit `src/description_generator/synthetic_tasks.py` to use same primitive names as description generator.

Key mappings needed:

```
# In SyntheticRule expected_primitives, use:
"n_unique_suits" instead of "unique"
"all_same_suit" instead of "uniform"
"count_suit" instead of "count"
```

Step 4: Re-run Evaluation (30 min)

```
python3 description_generator/evaluation_harness.py --output after_fix.json
```

Expected: FCR@3 should now be >0%

Step 5: Analyze Results

```
import json
with open('after_fix.json') as f:
    results = json.load(f)

# Check by level
for level, acc in results['by_level']['accuracy'].items():
    print(f"Level {level}: {acc:.1%}")
```

Success Criteria

Metric	Minimum	Good	Excellent
FCR@3 (Level 1-2)	30%	50%	70%
FCR@3 (Level 3-4)	20%	40%	60%
FCR@3 (Level 5/OOD)	10%	25%	40%
Consistency	60%	80%	90%
Discrimination	50%	70%	85%
vs Random Baseline	+10%	+20%	+30%

Debugging Tips

If all descriptions are “losing hands often: has 3 different suits”

This means the task examples don’t provide clear signal. Check: - Are positive/negative sets well separated?
- Is the rule too rare (can’t sample positives)?

If descriptions don’t mention expected feature

Check the DescriptionVocabulary in `description_generator.py` - there may not be a template for that feature type.

If consistency is low

The description generator may be sensitive to example selection. Consider: - Using more examples per rule
- Adding noise tolerance in vocabulary selection

Demo Commands

Describe a single hand

```
from description_generator.description_generator import DescriptionGenerator
from rules.cards import sample_hand

gen = DescriptionGenerator(n_baseline_samples=5000)
hand = sample_hand(6)
print(f"Hand: {[str(c) for c in hand]}")

for desc in gen.describe_hand(hand, top_k=5):
    print(f"  {desc.text} (score: {desc.score:.2f})")
    print(f"    Primitives: {desc.primitives}")
```

Describe a task (distinguishing positive from negative)

```
from description_generator.description_generator import DescriptionGenerator
from rules.cards import Card, Suit, Rank, card_color, Color, sample_hand

gen = DescriptionGenerator(n_baseline_samples=5000)

# Rule: all same color
def uniform_color(h):
    return len(set(card_color(c) for c in h)) == 1
```

```
# Sample
positive = [h for h in [sample_hand(6) for _ in range(500)] if uniform_color(h)][:20]
negative = [h for h in [sample_hand(6) for _ in range(500)] if not uniform_color(h)][:20]

print(f"Positive: {len(positive)}, Negative: {len(negative)}")
for desc in gen.describe_task(positive, negative, top_k=5):
    print(f"  {desc.text} (score: {desc.score:.2f})")
```

Next Steps After Testing

1. **If evaluation works well:** Proceed to integrate with DreamCoder recognition network
 2. **If vocabulary issues persist:** Create a vocabulary mapping layer
 3. **If accuracy is too low:** Expand description templates or feature extractors
 4. **If consistency is low:** Add example aggregation before description generation
-

Document prepared for evaluation session on December 26-27, 2024